

SEMBOT · CSS ONBOARDING

CSS Onboarding — pełna dokumentacja ścieżek rejestracji

Od css.sembot.com do realnego onboardingu w Sembocie — krok po kroku, z prawdziwymi zrzutami ekranu, treścią maili i osią czasu. Dla developerów i zespołu biznes/CS.

proxy_sembot + sembot-laravel — stan na 2026-07-15

Spis treści

Kontekst produktowy

Oś czasu — kiedy co się dzieje

Analityka i atrybucja

Rozdział A — Ścieżka CSS Shop

Wybór profilu — GET /auth/css

Rejestracja — GET/POST /auth/register, POST /signup

Formularz sklepu — GET/POST /css/shop

Strona sukcesu — GET /css/shop/done

Wizard audytu Sembota — cel ścieżki

Rozdział B — Ścieżka CSS Agency

Wybór profilu — GET /auth/css (ten sam ekran co Shop krok 1)

Rejestracja — identyczny formularz, css_type=css-agency

Strona agencji — GET /css/agency

Po kliknięciu mailto — potwierdzenie + zmiana CTA

Wizard Sembota, krok 1 — GET /onboarding/start → /onboarding/domain

Bridge — tworzenie projektu w tle

Cel osiągnięty — realny panel Sembota (Marketer AI + audyt)

Rozdział C — Treści maili

CssShopWelcomeMail — powitanie po CSS Shop

CssShopNotifyMail — zgłoszenie do zespołu CS

CssAgencyWelcomeMail — powitanie agencji + aktywny bonus

Tabela plików kluczowych

Status MVP i otwarte pytania

Kontekst produktowy

CSS = Google Comparison Shopping Service. Dwa osobne produkty za tą samą rejestracją:

Produkt	Co to jest	Model
Sembot CSS	Darmowa integracja Google Shopping	Darmowy na zawsze dla sklepów; 1 rok gratis dla agencji (White Label), potem €100/mo per market. Niezależny od planu Sembot.
Sembot	Płatna platforma do zarządzania kampaniami	Google/Meta/Microsoft Ads, feed produktowy, automatyzacja, AI agent. 14 dni pełnego dostępu jako bonus za rejestrację przez CSS.

Mechanizm lejka (dlaczego to wszystko istnieje): CSS jest lead magnetem. User rejestruje się po darmowy CSS → dostaje 14-dniowy trial Sembota → ma szansę zobaczyć wartość narzędzia → konwertuje na płatny plan. Każda decyzja opisana w tym dokumencie (segmentacja Shop/Agency, timing aktywacji trialu, treść maili) wynika z tego jednego mechanizmu — CSS sam w sobie nie generuje przychodu, jest przynętą.

Punkt wejścia: `https://css.sembot.com` → `https://new.sembot.com/auth/css` → `https://new.sembot.com/auth/register?type=css-shop` lub `?type=css-agency`. Dwie ścieżki rejestracji różnicowane przez ukryte pole `css_type`, zapisywane na koncie jako `account_type` (enum `AccountType::CSS_SHOP / CSS_AGENCY`). Cel to zawsze `new.sembot.com` — `app.sembot.com` to osobny, nadal działający legacy frontend Angular, bez ścieżki CSS.

Uwaga językowa: UI ścieżki wspiera pl/en. Wszystkie 3 maile transakcyjne (Rozdział C) są **tylko po polsku**, niezależnie od locale usera — brak wariantu EN.

Dokument obejmuje CAŁY flow — od wyboru profilu, przez rejestrację i formularze specyficzne dla CSS, przez treść maili transakcyjnych, aż po faktyczne wejście do wizarda onboardingu Sembota (`/onboarding/audit`).

Oś czasu — kiedy co się dzieje

Ten sam moment "T+0" (submit formularza rejestracji) prowadzi do bardzo różnych osi czasu dla Shop i Agency w kwestii aktywacji CSS przez Google — trial Sembota startuje identycznie w obu ścieżkach, aktywacja samego CSS/White Label już nie.

Moment	Ścieżka Shop	Ścieżka Agency
T+0 (submit formularza / rejestracja)	Projekt + feed tworzone automatycznie. Mail do CS (e2) i welcome mail do usera (e1) wysyłane od razu. Trial Sembota aktywuje się natychmiast.	Brak automatycznego projektu (dopiero wizard onboardingu Sembota go tworzy). Trial Sembota aktywuje się natychmiast przy rejestracji — welcome mail (e3) wysyłany od razu, nie czeka na kontakt z CS.
T+0, jeśli user napisze mailem	—	CS dostaje zgłoszenie (mailto, bez potwierdzenia dostarczenia — patrz krok B4) z danymi do konfiguracji White Label.
Aktywacja trialu Sembota	T+0, automatycznie (<code>SubmitShop</code> → <code>CSSStartTrial</code> , plus ogólny mechanizm rejestracji).	T+0, automatycznie, przy rejestracji (<code>UserRepository::createAccount()</code>) — tak samo jak Shop.
CS aktywuje CSS w Google	T+24-48h.	T+2-4 tygodnie od kompletnych danych.
Koniec 14-dniowego trialu Sembota	T+14 dni — z zapasem ~12 dni nakładania się z aktywnym CSS.	T+14 dni od rejestracji . Znane ryzyko: może wygasnąć zanim White Label CSS w ogóle ruszy (T+14-28 dni) — świadomie zaakceptowany kompromis na rzecz spójnej, jednolitej polityki trialu dla wszystkich kont.
Dostęp do platformy Sembot	Pełny, od T+0 — niezależnie od statusu trialu (w kodzie nie ma dziś egzekwowanego paywalla).	Pełny, od T+0 — identycznie jak Shop.

"Dostęp do platformy" w tabeli wyżej jest pełny od razu w OBU ścieżkach, niezależnie od kolumny "Aktywacja trialu Sembota" — bo w kodzie nie ma dziś żadnego middleware faktycznie egzekwującego trial/subskrypcję. "Aktywacja trialu" jest dziś polem informacyjno-raportowym, nie bramką dostępu.

Analityka i atrybucja

Kontekst: `css.sembot.com` planuje wersje językowe dla wszystkich krajów UE, ale Sembot realnie wspiera dziś UI tylko w `en / pl`. Niewspierane języki muszą trafiać do Sembota jako `en` — inaczej rejestracja skończy się twardym błędem 422 (`RegisterRequest` waliduje `language` przeciwko enumowi `EN/DE/PL`). Żeby nie stracić informacji "skąd faktycznie przyszedł lead" przy takim wymuszonym mapowaniu, istnieje osobne pole `source`, niezależne od `language`.

`?source=` (np. `css-es`) jest łapane na dowolnej stronie funnела i zapisywane na `User.source` (Laravel). Event PostHog `user_registered` — odpalany dla KAŻDEJ rejestracji w Sembocie, nie tylko CSS — jest wzbogacony o `language / account_type` (person property, `$set`) i `source` (atrybucja pierwszego dotknięcia, `$set_once` — nie nadpisywana przez późniejsze eventy).

Scenariusz	Przykładowy event <code>user_registered</code> (<code>\$set / \$set_once</code>)
Zwykła rejestracja (nie-CSS)	<code>language: "pl", account_type: null, source: null</code>
CSS Shop, lead z hiszpańskiej wersji <code>css.sembot.com</code>	<code>language: "en", account_type: "css_shop", source: "css-es"</code>
CSS Agency, lead z niemieckiej wersji	<code>language: "en", account_type: "css_agency", source: "css-de"</code>

Dzięki temu w PostHogu da się odpowiedzieć na pytanie "ile rejestracji przyszło z każdego języka, i czy konwertują tak samo dobrze jak `en/pl`" — bez ręcznego joinowania danych z bazy, bo `source / language / account_type` są trwale zapisane na profilu osoby i dziedziczą je też wszystkie późniejsze eventy tego usera (`subscription_created` itd.).

Jak source i lang przeżywają cały funnel

`middleware.WithSource` (`internal/middleware/source.go`) i `middleware.WithLocale` (`internal/middleware/i18n.go`) są wpięte globalnie w `chain` (`cmd/server/main.go`) — działają na KAŻDEJ stronie, nie tylko `/auth/register`. Dzięki temu `source` i `lang` przeżywają oba warianty linkowania z `css.sembot.com`: czy to wspólny wybierak profilu (`/auth/css`, którego karty mają zahardkodowane hrefy bez własnego query stringa), czy bezpośredni link do konkretnej ścieżki (`/auth/register?type=...`).

- **Persystencja:** oba parametry, raz złapane, trafiają do cookie (`sembot_source`, `sembot_locale`) — kolejne strony w tym samym funnelu nie muszą powtarzać query stringa.
- **Kolejność źródeł:** `ShowSignup / DoSignup` czytają przez `middleware.Source(r) / middleware.Locale(r)`, więc działa niezależnie czy user trafia na `/auth/register` bezpośrednio, czy przez `/auth/css`.
- **Walidacja source :** wolny tekst, jedyna reguła to regex `^[a-zA-Z0-9-]{1,64}$` (`SanitizeSource`) — litery/cyfry/myślnik, max 64 znaki. Brak rejestru/allowlisty w PostHogu — co zostanie wysłane, to 1:1 (case-sensitive) trafia jako wartość pola.
- **Niewspierany ?lang= :** mapuje się deterministycznie na domyślny język (`en`) i jest persystowany w cookie — ten sam link zawsze daje ten sam efekt, niezależnie od przeglądarki leada (nie zależy od nagłówka `Accept-Language`).
- `lang=` przełącza cały widoczny UI formularza rejestracji, nie tylko atrybut `language` na koncie — resolved locale idzie przez `ui.WithUI(ctx, loc, ...)` do każdego `ui.T(ctx, key)` w szablonach.

First-touch vs last-touch

`lang` i `source` mają świadomie różną semantykę nadpisywania cookie:

Parametr	Zasada
----------	--------

lang	Last-touch — każdy kolejny <code>?lang=</code> nadpisuje cookie. User może świadomie przełączyć język w dowolnym momencie.
source	First-touch — raz zapisany, nie jest nadpisywany przez kolejne <code>?source=</code> (cookie żyje 30 dni). Atrybucja ma wskazywać PIERWSZE dotknięcie leada, nie ostatni klik.

Konsekwencja: user, który odwiedzi PL, potem EN, potem zarejestruje się, wyląduje z `language=en` (ostatni klik) ALE `source=css-pl` (pierwszy klik). To zamierzone rozjechanie dla poprawności atrybucji, nie błąd — `lang` i `source` nie zawsze opisują tę samą wizytę dla użytkowników przełączających język w trakcie sesji.

Linki wejściowe z **css.sembot.com**

Aktualna lista linków wdrożonych po stronie **css.sembot.com**:

Wersja	Element	URL
PL	nav-CTA (topbar, wybór profilu)	<code>new.sembot.com/auth/css?lang=pl&source=css-pl</code>
EN	nav-CTA (topbar, wybór profilu)	<code>new.sembot.com/auth/css?lang=en&source=css-en</code>
PL	CTA "Sklep" (/ecommerce, profil znany)	<code>new.sembot.com/auth/register?type=css-shop&lang=pl&source=css-pl</code>
EN	CTA "Sklep" (/ecommerce, profil znany)	<code>new.sembot.com/auth/register?type=css-shop&lang=en&source=css-en</code>
PL	CTA "Agencja" (/agency, profil znany)	<code>new.sembot.com/auth/register?type=css-agency&lang=pl&source=css-pl</code>
EN	CTA "Agencja" (/agency, profil znany)	<code>new.sembot.com/auth/register?type=css-agency&lang=en&source=css-en</code>

Tenant **css.sembot.com** ma dziś tylko 2 wersje językowe (EN root, PL `/pl`). Wzorec dla przyszłych niewspieranych języków UE (es, de, fr, it, nl...) — `<kod_języka>` to dwuliterowy kod ISO wersji strony:

- Jeśli dana wersja nie rozróżnia jeszcze Sklep/Agencja: `new.sembot.com/auth/css?lang=en&source=css-<kod_języka>`
- Jeśli ma już osobne CTA: `new.sembot.com/auth/register?type=css-shop&lang=en&source=css-<kod_języka>` / `type=css-agency` odpowiednio

Oba warianty działają poprawnie — nie trzeba rozróżniać, przez który punkt wejścia trafia lead.

Dlaczego `source` jest ustawiane też dla PL/EN, mimo że jest tam redundantne: dla wspieranych języków `source=css-pl` / `css-en` nie niesie żadnej nowej informacji ponad to, co już dają `account_type` (jednoznacznie: to lead z CSS) + natywne `language` (jednoznacznie: wersja językowa) — w przeciwieństwie do niewspieranych języków, gdzie `language` jest wymuszane na `en` i `source` jest jedynym źródłem prawdy o faktycznym pochodzeniu. Mimo tej redundancji `source` zostaje wypełnione wszędzie: koszt jest zerowy, a alternatywa (`source` puste dla większości ruchu CSS) tworzy realne ryzyko — analityk filtrujący "leady z CSS" przez `source LIKE 'css-%'` cicho zaniżyłby wynik o całą populację PL/EN, bez żadnego błędu czy ostrzeżenia.

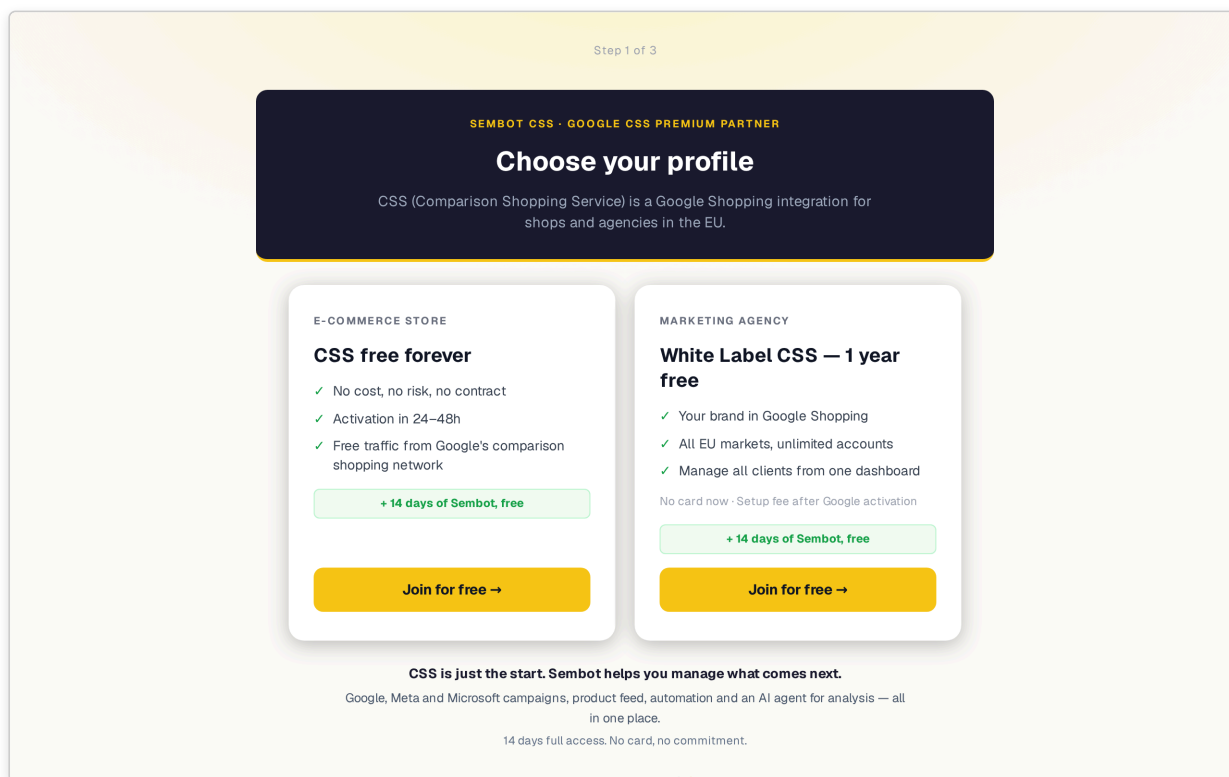
Rozdział A — Ścieżka CSS Shop

auth/css → register?type=css-shop → /css/shop → /css/shop/done → onboarding/audit

1 Wybór profilu — GET /auth/css

Trasa: GET /auth/css

Handler: Auth.ShowCSSPath — internal/handlers/auth.go:108



01-wybor-profilu.png

CO WIDZI UŻYTKOWNIK

Ciemna karta hero (spójna z css.sembot.com) + dwie karty wyboru, wizualnie równorzędne: "E-commerce Store" i "Marketing Agency". Pod kartami blok wyjaśniający czym jest Sembot (Google/Meta/Microsoft, feed, AI agent, 14 dni bez karty).

KONTEKST BIZNESOWY

CSS to darmowy lead magnet — segmentacja Shop/Agency na starcie pozwala dopasować dalej model przychodu do właściwego typu klienta: sklep dostaje CSS za darmo na zawsze, agencja rok gratis a potem €100/mies. za rynek. To inny model monetyzacji, więc komunikat na tej stronie jasno rozróżnia oba segmenty.

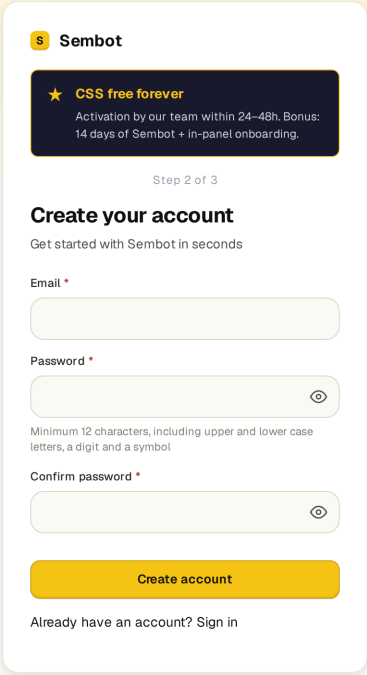
CO SIĘ DZIEJE POD MASKĄ

- Handler renderuje statyczną stronę wyboru — brak zapisu do sesji ani wywołań API na tym etapie.
- CTA "Join for free ->" na karcie Sklep prowadzi do `/auth/register?type=css-shop`; karta Agencja do `/auth/register?type=css-agency`.
- Blok wyjaśniający Sembota (`.css-path-sembot-info`) tłumaczy że CSS to dopiero początek — pełna platforma obejmuje Google/Meta/Microsoft, feed produktowy, automatyzację i AI agenta.
- Atrybucja źródła leada (`?source=`) i język (`?lang=`) przeżywają przejście przez ten ekran do rejestracji, mimo że karty mają zahardkodowane hrefy bez własnego query stringa — mechanizm opisany w rozdziale "Analityka i atrybucja".

2 Rejestracja — GET/POST /auth/register, POST /signup

Trasa: GET /auth/register?type=css-shop → POST /signup

Handler: Auth.ShowSignup (signup.go:25) / Auth.DoSignup (signup.go:50)



02-rejestracja-shop.png

CO WIDZI UŻYTKOWNIK

Ciemny baner nad formularzem: "CSS bezpłatny na zawsze — Aktywacja przez nasz zespół w 24-48h. Bonus: 14 dni Sembota + onboarding w panelu." Standardowy formularz: email + hasło (walidacja inline Alpine: min. 12 znaków, wielka/mala litera, cyfra, symbol) + potwierdzenie hasła.

KONTEKST BIZNESOWY

Rejestracja to jedyny moment, w którym zapisujemy dane decydujące o dalszej monetyzacji konta (`account_type`). Baner nad formularzem musi jasno komunikować że to rejestracja do konkretnego programu CSS, nie generyczne konto Sembot.

CO SIĘ DZIEJE POD MASKĄ

- `ShowSignup` czyta `?type=` z query, sanityzuje do `css-shop / css-agency /pusty` (nieznana wartość = zwykła rejestracja) i przekazuje jako ukryte pole `css_type` w formularzu.
- `DoSignup` (POST /signup, HTMX) waliduje zgodność haseł lokalnie, woła Laravel `POST /api/v1/register` (`AccountType` mapowane z myślnika na podkreślnik po stronie Lavela — `LoginController.php:180`), następnie auto-loguje przez `POST /login` żeby od razu mieć JWT.
- Destynacja po sukcesie zależy od `css_type`: `css-shop` → `/css/shop`, `css-agency` → `/css/agency`, inne → `/onboarding/start` (signup.go:108-113).
- Formularz przyjmuje też `?source=` (np. `css-es`) — ukryte pole analogiczne do `css_type`, sanityzowane i przenoszone do `User.source` w Laravelu. Czysto atrybucyjne, niewidoczne w UI — patrz rozdział "Analityka i atrybucja".

3 Formularz sklepu — GET/POST /css/shop

Trasa: GET /css/shop → POST /css/shop

Handler: CSSOnboarding.ShowShop (css_onboarding.go:37) / CSSOnboarding.SubmitShop (css_onboarding.go:52)

Step 3 of 3

SEMBOT CSS - GOOGLE CSS PREMIUM PARTNER

Your CSS is free forever

Fill in the details — our team will activate your CSS within 24–48h.

★ BONUS: 14 days of Sembot + in-panel onboarding

Store name *

Testowy Sklep PDF

Product feed URL *

https://testowy-sklep-pdf.example.com/feed.xml

Product feed URL from your store. Shopify: Sales Channels → Google → Feed URL. WooCommerce: Product Feed plugin. PrestaShop: Modules → Google Merchant Center.

Store domain *

testowy-sklep-pdf.example.com

GMC ID (Google Merchant Center) (optional)

123456789

Where to find your GMC ID? Log in to merchants.google.com → number in the top-left or in the URL (?a=XXXXXXXXX). No account? [Create account](#) — takes about 10 minutes.

04b-formularz-wypelniony.png

CO WIDZI UŻYTKOWNIK

Pola: nazwa sklepu (wymagane), URL feeda produktowego (wymagane, z podpowiedzią Shopify/WooCommerce/PrestaShop), domena sklepu (wymagane, auto-fill z hosta feed URL przez Alpine `x-on:input`), GMC ID (opcjonalne, domyślnie zwinięte za linkiem-togglęm, z instrukcją gdzie znaleźć w Merchant Center). CTA: "Activate CSS and go to your panel".

KONTEKST BIZNESOWY

Minimalny formularz (3 pola wymagane) obniża barierę wejścia na etapie, na którym user jeszcze nie zaangażował się w produkt. GMC ID jest domyślnie zwinięte, bo część sklepów jeszcze go nie ma na starcie — wymuszanie go od razu realnie blokowałoby konwersję.

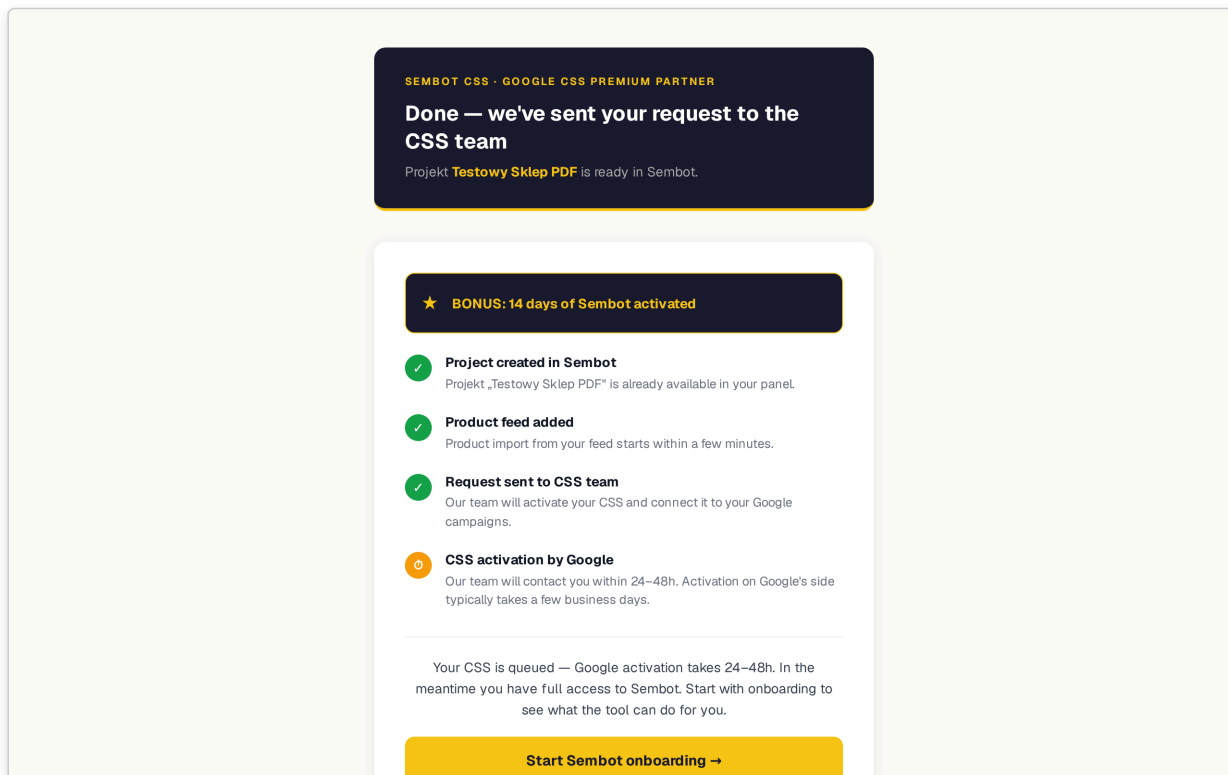
CO SIĘ DZIEJE POD MASKĄ

- `SubmitShop` woła sekwencyjnie: `CSSCreateProject` (POST `/api/v1/css/shop/project` — auto-weryfikuje email usera server-side, tworzy projekt typu `SIMPLE`), `CreateInputFeed` (feed z domyślnym mapowaniem 18 pól Google Shopping, `api.DefaultCSSFeedMapping()`), `CSSShopNotify` (POST `/api/v1/css/shop/notify` — wysła DWA maile naraz, patrz Rozdział C), `CSSStartTrial` (POST `/api/v1/css/start-trial` — aktywuje 14-dniowy trial natychmiast).
- Błędy `CreateInputFeed`, `CSSShopNotify` i `CSSStartTrial` są nieblokujące (tylko `log.Printf`) — projekt i przekierowanie na success page następują nawet gdy feed/notify/trial zawiodą. Tylko błąd `CSSCreateProject` zatrzymuje flow i re-renderuje formularz z komunikatem błędu.
- **Ograniczenie:** jeśli landlord DB nie ma żadnego wiersza w tabeli `tenants`, `CSSCreateProject` zwraca 500 (`StoreProject.php:89` — brak gałęzi na „zero tenantów”). Dotyczy tylko środowisk bez zaimportowanego snapshotu z realnymi tenantami.

4 Strona sukcesu — GET /css/shop/done

Trasa: GET /css/shop/done?ws=&proj=&name=

Handler: CSSOnboarding.ShowShopDone (css_onboarding.go:142)



05-potwierdzenie-shop.png

CO WIDZI UŻYTKOWNIK

Checklistka: ✓ Projekt stworzony, ✓ Feed dodany, ✓ Zgłoszenie do CS, ⌚ Aktywacja CSS przez Google (24-48h). Bonus box "BONUS: 14 days of Sembot activated". CTA primary: "Start Sembot onboarding →", secondary: link do panelu projektu.

KONTEKST BIZNESOWY

To jest właściwy cel całego lejka, nie CSS samo w sobie: user dostał darmowy CSS, ale ta strona pokazuje mu że dostaje też 14 dni Sembota — płatnej platformy. Ten moment decyduje, czy user w ogóle spróbuje Sembota, czy tylko odbierze CSS i zniknie. Aktywacja CSS Shop trwa 24-48h, czyli wygodnie mieści się w 14-dniowym trialu (w Agencji ta relacja wygląda inaczej — aktywacja White Label trwa 2-4 tygodnie, patrz Rozdział B).

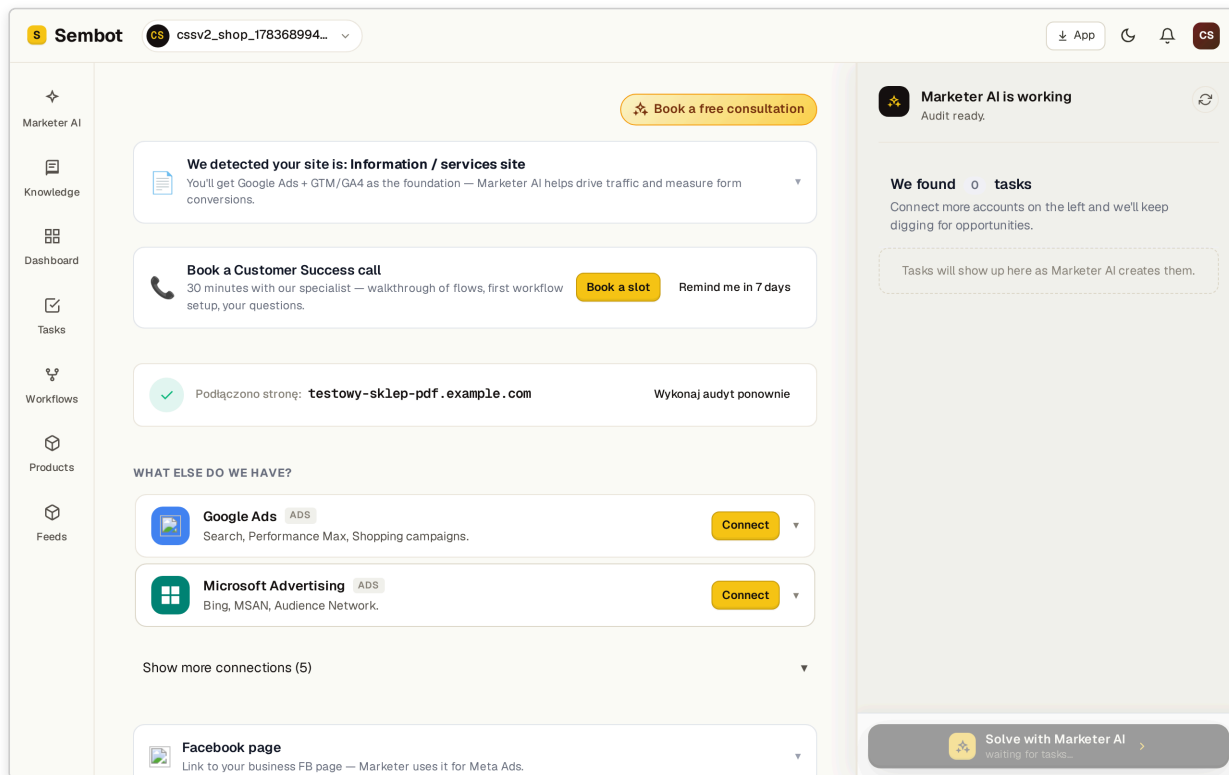
CO SIĘ DZIEJE POD MASKĄ

- Parametry `ws / proj / name` idą przez query string; sesja jest i tak weryfikowana przez `requireSession`.
- Primary CTA linkuje na `/p/{ws}/{proj}/onboarding/audit` — scoped route, bezpośrednio do wizarda audytu Sembota.
- `CssShopWelcomeMail` (osobny mail DO USERA, nie do CS) jest wysyłany z `CssOnboardingController::shopNotify()` gdy request zawiera `project_url` — pełna treść w Rozdziale C.

5 Wizard audytu Sembota — cel ścieżki

Trasa: GET /p/{ws}/{proj}/onboarding/audit (kliknięcie CTA z kroku 4)

Handler: Onboarding.ShowAudit (onboarding.go:2008)



08-onboarding-audit-shop.png

CO WIDZI UŻYTKOWNIK

Po kliknięciu CTA użytkownik widzi pełny panel Sembota: Marketer AI, lista integracji do podłączenia (Google Ads, Microsoft Advertising, Facebook...), wynik audytu strony.

KONTEKST BIZNESOWY

To jest realny cel całego lejka CSS→Sembot — user, który dotarł tutaj, widzi prawdziwą wartość płatnej platformy, nie tylko darmowy CSS.

CO SIĘ DZIEJE POD MASKĄ

- `SubmitShop` utrzuła scope nowego projektu (`ProjectSlug` , `OnboardingStep`) w sesji zaraz po `CSSCreateProject` , więc kliknięcie CTA z kroku 4 prowadzi bezpośrednio tutaj, a nie na listę zadań.
- `audit.RunAndPersist` (wywoływane w `ShowAudit`) automatycznie tworzy pierwsze taski na bazie wyniku audytu strony.

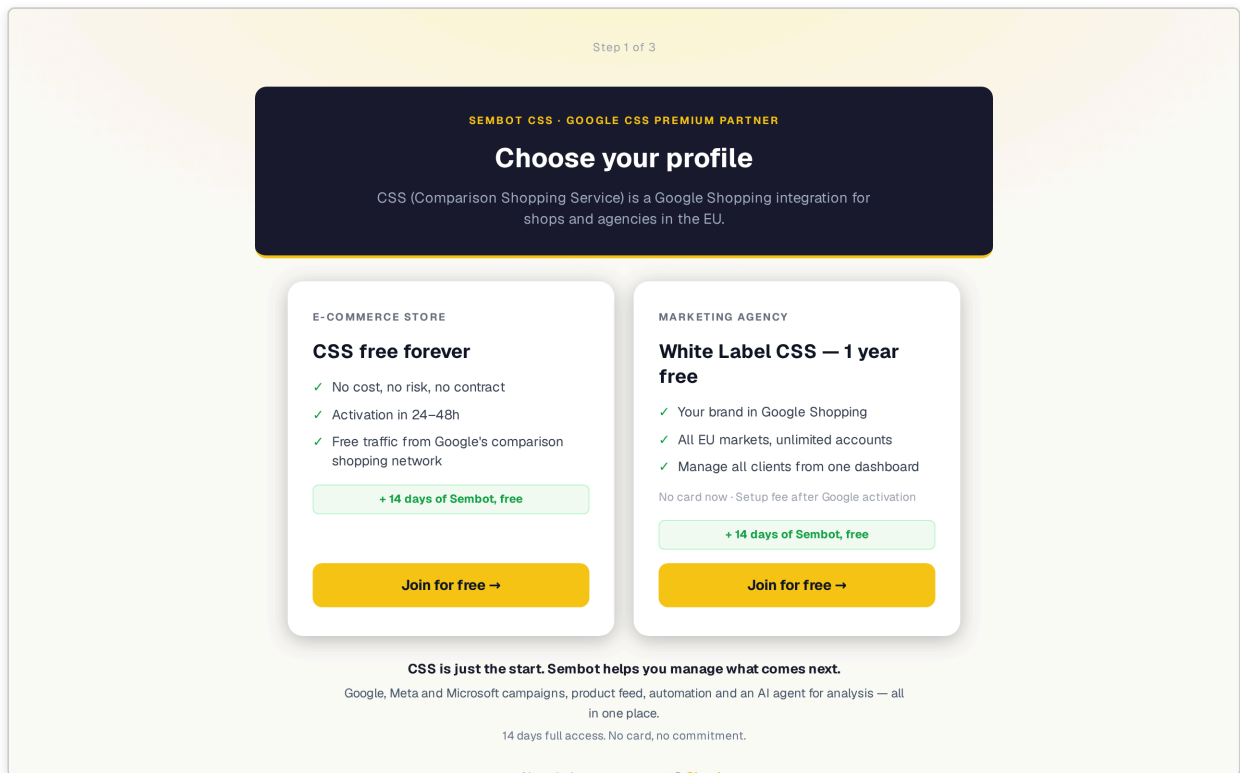
Rozdział B — Ścieżka CSS Agency

auth/css → register?type=css-agency → /css/agency → (mailto) → onboarding/start → onboarding/domain → onboarding/audit-bridge → onboarding/audit

1 Wybór profilu — GET /auth/css (ten sam ekran co Shop krok 1)

Trasa: GET /auth/css

Handler: Auth.ShowCSSPath — internal/handlers/auth.go:108



01-wybor-profilu.png (ekran wspdzielony z pozostała ścieżka)

CO WIDZI UŻYTKOWNIK

Identyczny ekran jak w ścieżce Shop — użytkownik klika kartę "Marketing Agency" zamiast "E-commerce Store".

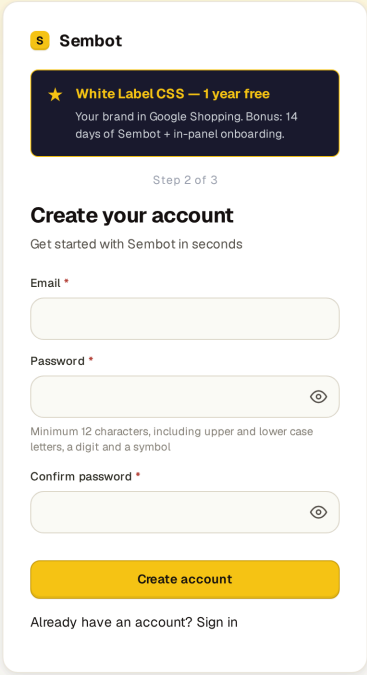
CO SIĘ DZIEJE POD MASKĄ

- CTA karty Agencja prowadzi do /auth/register?type=css-agency (zamiast css-shop) — to jedyna różnica względem ścieżki Shop na tym etapie. Kontekst biznesowy — patrz Rozdział A, krok 1.

2 Rejestracja — identyczny formularz, `css_type=css-agency`

Trasa: GET `/auth/register?type=css-agency` → POST `/signup`

Handler: `Auth.ShowSignup` / `Auth.DoSignup` (`signup.go:25, 50`)



The screenshot shows a registration form for Sembot. At the top, there's a banner for 'White Label CSS — 1 year free'. Below it, the form is titled 'Create your account' and indicates it's 'Step 2 of 3'. The form includes three input fields: 'Email', 'Password', and 'Confirm password', each with a toggle for visibility. A yellow 'Create account' button is at the bottom. Below the button, there's a link: 'Already have an account? Sign in'.

03-rejestracja-agency.png

CO WIDZI UŻYTKOWNIK

Ten sam formularz email/hasło co Shop, z bannerem dopasowanym do agencji.

KONTEKST BIZNESOWY

Ten sam formularz co Shop, ale zapisane tu `account_type=css_agency` uruchamia INNY model przychodu (rok gratis + €100/mies. per rynek, zamiast darmo na zawsze) — to biznesowo najważniejsza rzecz zbierana na tym ekranie, mimo że wizualnie nic się nie różni.

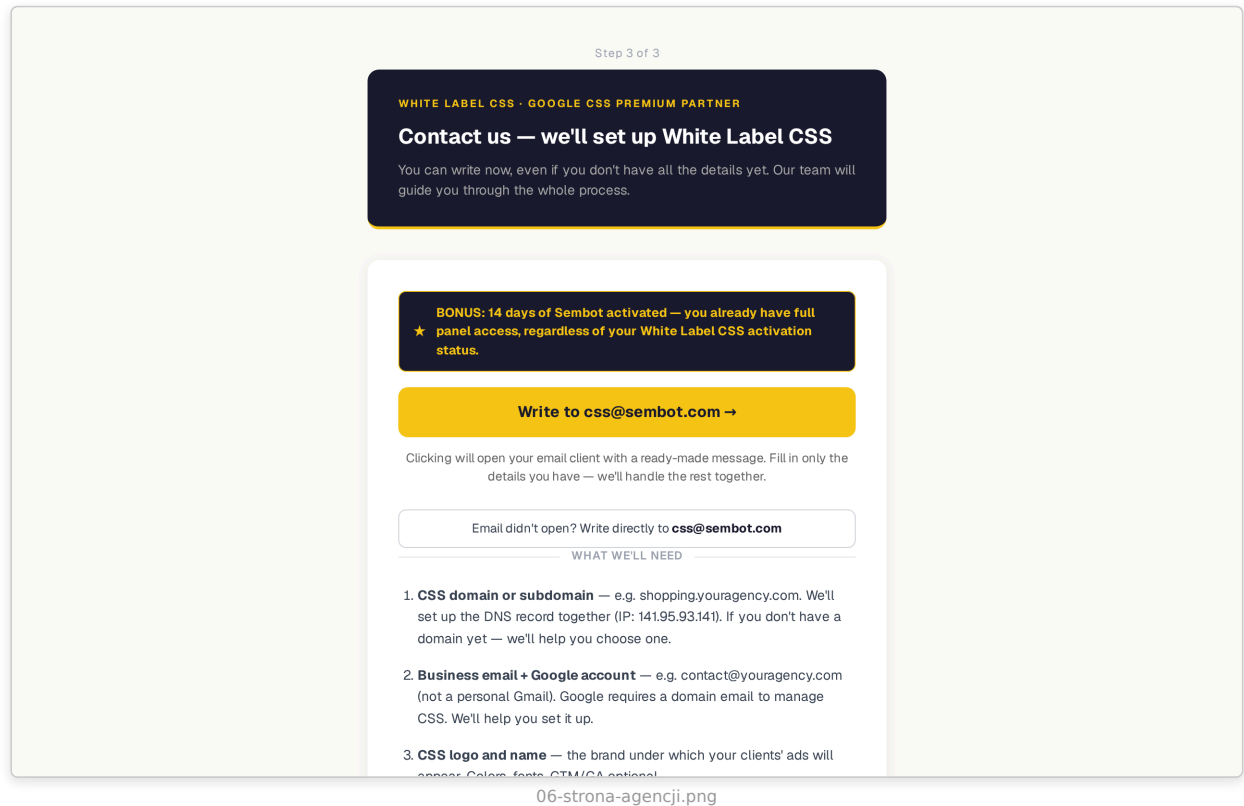
CO SIĘ DZIEJE POD MASKĄ

- Jedyna różnica w `DoSignup` względem ścieżki Shop: destynacja po rejestracji to `/css/agency` zamiast `/css/shop` (`signup.go:111`).
- `account_type = AccountType::CSS_AGENCY` jest zapisywany na modelu `User` i decyduje o modelu przychodu oraz o treści maila powitalnego (Rozdział C).
- `?source=` działa identycznie jak w kroku 2 ścieżki Shop (ten sam handler).

3 Strona agencji — GET /css/agency

Trasa: GET /css/agency

Handler: CSSOnboarding.ShowAgency (css_onboarding.go:194)



CO WIDZI UŻYTKOWNIK

Hero "Contact us — we'll set up White Label CSS". Box: "BONUS: 14 days of Sembot activated — you already have full panel access, regardless of your White Label CSS activation status." Lista wymagań (domena/subdomena CSS, email domenowy + konto Google, logo i nazwa) widoczna zawsze. Duży żółty przycisk "Write to css@sembot.com →" (prefillowany mailto).

KONTEKST BIZNESOWY

Segmentacja Agency uruchamia inny model przychodu niż Shop (rok gratis + €100/mies. per rynek zamiast darmo na zawsze) i inny harmonogram: aktywacja White Label przez Google trwa 2-4 tygodnie, dłużej niż 14-dniowy trial Sembota. To znany, świadomie zaakceptowany kompromis — trial startuje razem z rejestracją, żeby polityka produktu była spójna dla wszystkich kont, mimo że w tej konkretnej ścieżce może wygasnąć zanim White Label ruszy.

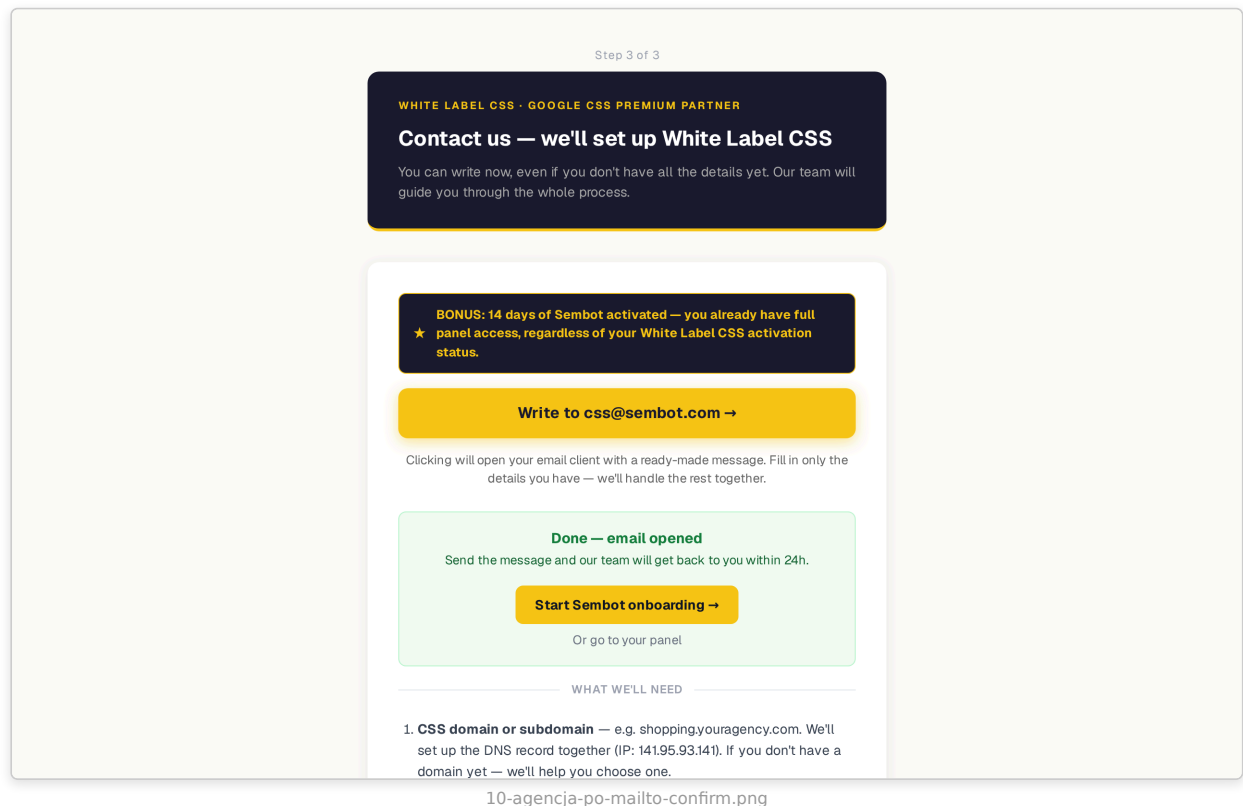
CO SIĘ DZIEJE POD MASKĄ

- Trial Sembota aktywuje się automatycznie przy rejestracji (`UserRepository::createAccount()` , Laravel) — tak samo jak dla każdego innego konta, niezależnie od `account_type` .
- `CssAgencyWelcomeMail` wysyłany w tym samym miejscu, od razu przy rejestracji — pełna treść w Rozdziale C.
- W kodzie nie ma dziś egzekwowanego paywalla nigdzie w produkcie — "aktywacja trialu" jest polem informacyjno-raportowym, nie bramką dostępu. User ma pełny dostęp do platformy od razu po rejestracji, niezależnie od statusu trialu.
- Dodatkowe narzędzia dla zespołu CS: `CssOnboardingController::startTrial()` (`POST /api/v1/css/start-trial`) i `SubscriptionController::extendTrial()` (`POST /users/{user}/subscription/trial/extend` , superadmin) pozwalają ręcznie re-aktywować lub przedłużyć trial, np. gdy aktywacja White Label się opóźni.

4 Po kliknięciu mailto — potwierdzenie + zmiana CTA

Trasa: `client-side` (Alpine.js, brak requestu do serwera)

Handler: `css_onboarding_agency.templ:52-72` — `x-data="{emailClicked: false}"`



CO WIDZI UŻYTKOWNIK

Po kliknięciu "Write to `css@sembot.com` →" (otwiera klienta poczty, `x-on:click="emailClicked=true"`): zielony box "Done — email opened" + nowe CTA "Start Sembot onboarding →" (primary) i "Or go to your panel" (secondary, szary link).

KONTEKST BIZNESOWY

mailto jako jedyna droga zgłoszenia to świadomy kompromis kosztowy zakresu — formularz z bezpośrednim zapisem do systemu jest odłożony poza zakres. Ryzyko biznesowe: serwer nie wie czy user faktycznie wysłał wiadomość, więc część leadów może się zgubić bez żadnego śladu w systemie — CS dowiaduje się o zgłoszeniu tylko jeśli mail faktycznie dotarł.

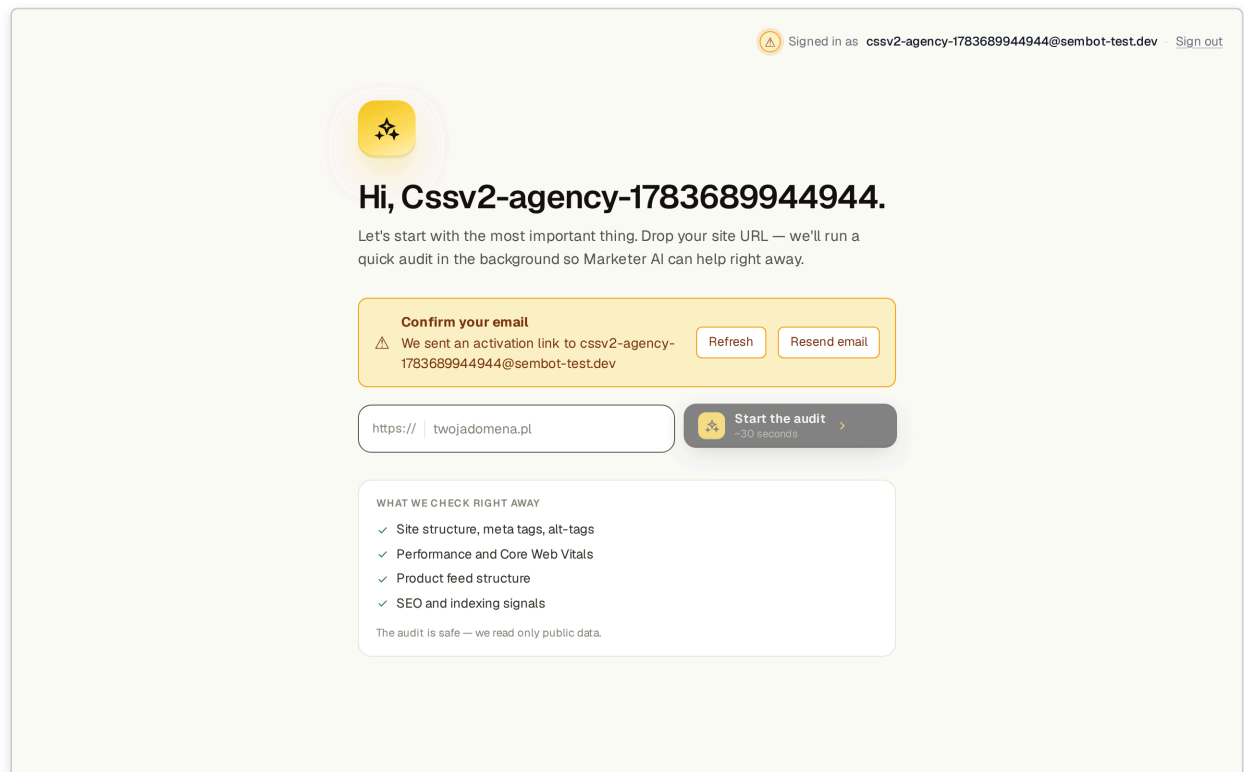
CO SIĘ DZIEJE POD MASKĄ

- Primary CTA po kliknięciu mailto prowadzi na `/onboarding/start` — bezpośrednio do wizarda Sembota.
- To jest CZYSTO client-side toggle — mailto: nie generuje żadnego requestu HTTP który serwer mógłby zaobserwować; system nie wie czy user faktycznie wysłał email.
- Przycisk "Start Sembot onboarding" nie zależy od statusu trialu (`/onboarding/start` nie sprawdza subskrypcji/trialu) — user może wejść do platformy natychmiast.

5 Wizard Sembota, krok 1 — GET /onboarding/start → /onboarding/domain

Trasa: GET /onboarding/start → 303 → GET /onboarding/domain

Handler: Onboarding.Start (onboarding.go:125) → Onboarding.ShowDomain (onboarding.go:197)



11-onboarding-domain-agency.png

CO WIDZI UŻYTKOWNIK

"Hi, {imię}. Let's start with the most important thing. Drop your site URL." Pole domeny + CTA "Start the audit (~30 seconds)". Widoczny żółty banner "Confirm your email — we sent an activation link to {email} [Refresh] [Resend email]" dopóki email nie jest zweryfikowany.

KONTEKST BIZNESOWY

Agencja, w przeciwieństwie do Shopu, nie ma automatycznie tworzonego projektu przy rejestracji — dopiero wejście do wizarda Sembota to pierwszy realny sygnał zaangażowania w platformę, niezależnie od tego czy/kiedy CSS się aktywuje.

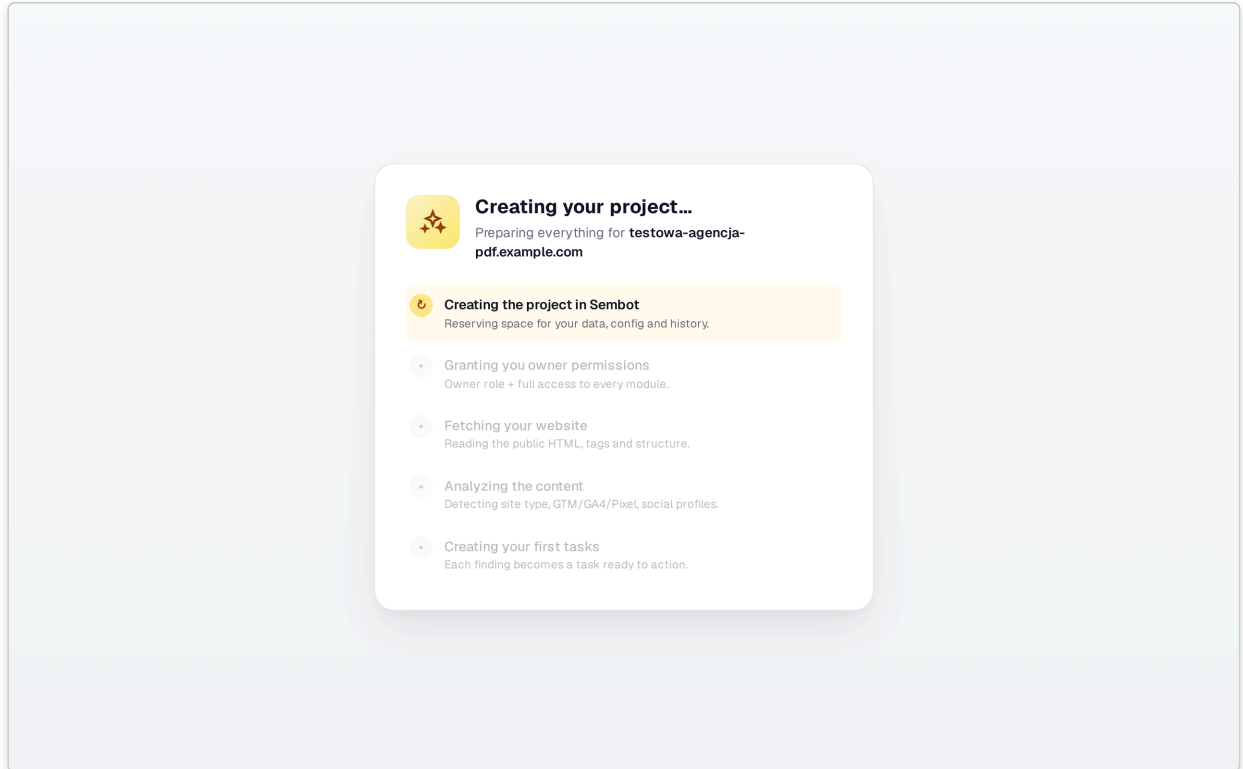
CO SIĘ DZIEJE POD MASKĄ

- Start sprawdza `firstProjectDest()` — agencja świeżo zarejestrowana NIE MA projektu (w przeciwieństwie do Shop, gdzie `CSSCreateProject` tworzy projekt od razu), więc trafia na krok 1 wizarda (`stepDomain`) zamiast na `/tasks` .
- `SaveDomain` (POST, `onboarding.go:257-282`) blokuje submit dopóki email nie jest zweryfikowany — dla ścieżki `Agency email` NIE jest auto-weryfikowany server-side (w przeciwieństwie do Shop, gdzie `createShopProject` jawnie woła `markEmailAsVerified()`). Agencja musi kliknąć prawdziwy link weryfikacyjny z emaila zanim wizard przepuści submit domeny.
- **Znane ograniczenie:** link weryfikacyjny w mailu (`{new_front_url}/auth/verify/{token}`), generowany przez `App\Notifications\VerifyEmail::verificationUrl()` wskazuje na trasę, której proxy_sembot dziś nie rejestruje — użytkownik klikający ten link trafia na 404 zamiast zostać zweryfikowanym.
- Banner ma dwie akcje: "Refresh" (sprawdź czy już zweryfikowano) i "Resend email" (POST /onboarding/resend-verify , ponowna wysyłka gdy mail nie dotarł).

6 Bridge — tworzenie projektu w tle

Trasa: POST /onboarding/domain → 303 → GET /onboarding/audit-bridge (polling GET ../status)

Handler: Onboarding.SaveDomain → Onboarding.ShowAuditBridge (onboarding.go:2808) / BridgeStatus (onboarding.go:2839)



12-onboarding-bridge-agency.png

CO WIDZI UŻYTKOWNIK

Checklist z żywym statusem: "Creating the project in Sembot" (aktywny spinner) → "Granting you owner permissions" → "Fetching your website" → "Analyzing the content" → "Creating your first tasks" (kolejne kroki wyszarzone, czekają).

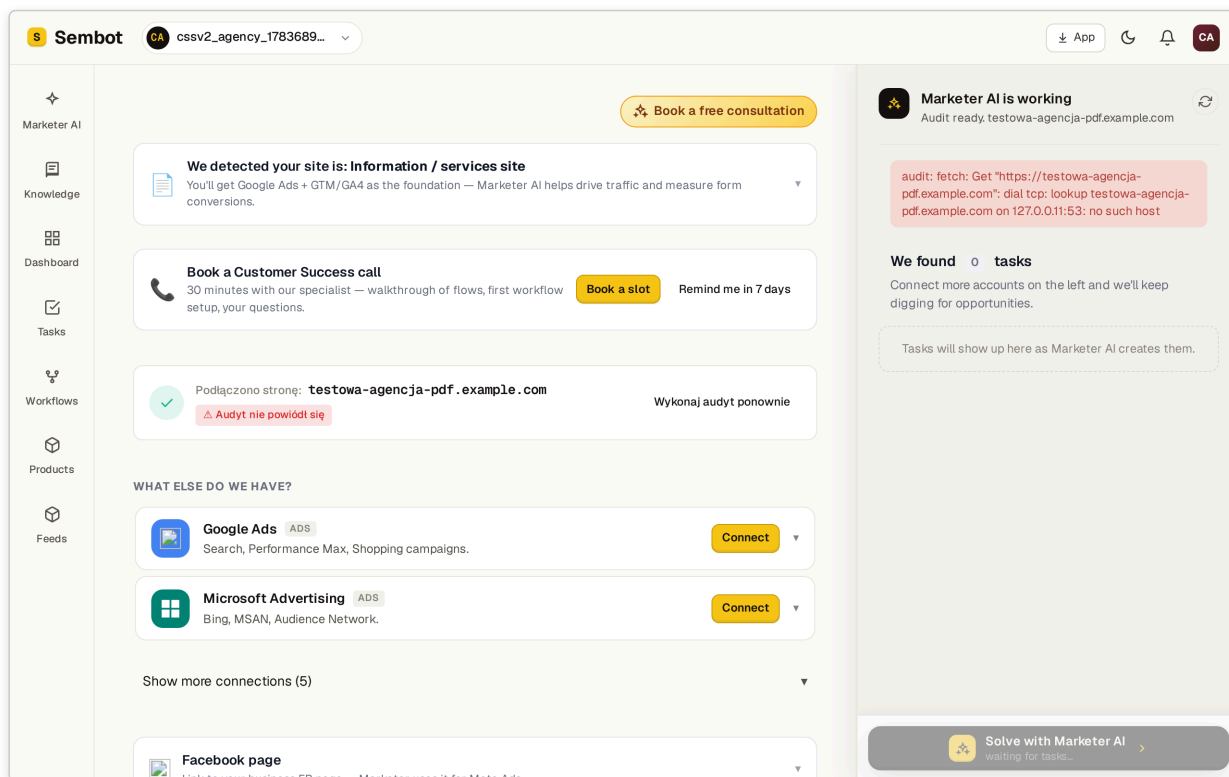
CO SIĘ DZIEJE POD MASKĄ

- SaveDomain odpala tworzenie projektu w goroutine i OD RAZU przekierowuje na bridge — projekt tworzy się asynchronicznie żeby user nie czekał ~11s (rola Owner + ~100 uprawnień) na zablokowanym submit.
- Bridge page (htmx hx-trigger="load delay:500ms, every 1000ms") pyta /onboarding/audit-bridge/status ; handler sprawdza czy projekt jest gotowy i zwraca header HX-Redirect na /p/{ws}/{proj}/onboarding/audit , zwykle w ciągu 2-4s.
- **Znana usterka:** automatyczne przekierowanie klienta (htmx polling) sporadycznie nie następuje mimo gotowego nagłówka po stronie serwera — mechanizm nie jest w pełni niezawodny pod obciążeniem.

7 Cel osiągnięty — realny panel Sembota (Marketer AI + audyt)

Trasa: GET /p/{ws}/{proj}/onboarding/audit

Handler: Onboarding.ShowAudit (onboarding.go:2008)



13-onboarding-audit-agency.png

CO WIDZI UŻYTKOWNIK

Pełny panel Sembota: sidebar (Marketer AI, Knowledge, Dashboard, Tasks, Workflows, Products, Feeds), lewa kolumna z wykrytym typem strony i listą integracji do podłączenia (Google Ads, Microsoft Advertising, Facebook...), prawy panel "Marketer AI is working" z wynikiem audytu.

KONTEKST BIZNESOWY

Ta sama wartość produktu co dla Shop — user, niezależnie którą ścieżką dotarł do projektu, trafia do prawdziwej platformy Sembot.

CO SIĘ DZIEJE POD MASKĄ

- To jest DOKŁADNIE ten sam `OnboardingAudit` widok co dla ścieżki Shop (krok 5/A5) — jeden szablon, jedna logika.
- `audit.RunAndPersist` (wywoływane w `ShowAudit`) automatycznie tworzy pierwsze taski na bazie wyniku audytu strony.

Rozdział C — Treści maili

3 maile transakcyjne w całym flow — pełna treść, trigger, odbiorca, kontekst biznesowy

1 CssShopWelcomeMail — powitanie po CSS Shop

Kiedy: `CssOnboardingController::shopNotify()` — TYLKO gdy request zawiera `project_url` (czyli `CSSCreateProject` wcześniej się powiodło)

Do kogo: User (email podany przy rejestracji)

Temat: Twój CSS jest w kolejce aktywacji — {shopName}

Zgłoszenie CSS przyjęte ✓

Projekt **{shopName}** jest gotowy w Sembocie. Nasz zespół aktywuje Twój CSS w ciągu **24-48 godzin**.

[[Przejdź do panelu projektu →](#)]

BONUS: 14 dni Sembota aktywowane

Zanim CSS zostanie aktywowany przez Google, możesz już teraz:

- ✓ Sprawdzić jak Twoje produkty wyglądają w panelu
- ✓ Skonfigurować kampanie Google Shopping
- ✓ Przeglądać i optymalizować produkty
- ✓ Projekt „{shopName}” stworzony w Sembocie
- ✓ Feed produktowy dodany — import zaczyna się w ciągu minut
- ✓ Zgłoszenie wysłane do zespołu CSS
- 🕒 Aktywacja CSS przez Google — nasz zespół skontaktuje się z Tobą w 24-48h

Pytania? Napisz na css@sembot.com — jesteśmy tu, żeby pomóc.

KONTEKST BIZNESOWY

Jedyny mail do usera w całej ścieżce Shop. Buduje poczucie że coś się realnie dzieje podczas 24-48h oczekiwania na aktywację CSS przez Google, i przypomina o bonusie Sembota (re-engagement gdyby user zamknął panel od razu po rejestracji).

2 CssShopNotifyMail — zgłoszenie do zespołu CS

Kiedy: `CssOnboardingController::shopNotify()` — ZAWSZE przy submisie formularza sklepu, niezależnie czy projekt się utworzył

Do kogo: `css@sembot.com` (zespół CS — NIE user)

Temat: CSS Shop Onboarding — {shopName} / GMC: {gmclId}

Nowe zgłoszenie CSS Shop Onboarding

Nazwa sklepu: {shopName}
GMC ID: {gmclId}
URL feeda: {feedUrl}
Domena: {domain}
Email użytkownika: {user.email}
Projekt w Sembocie: {projectUrl}

KONTEKST BIZNESOWY

To jest realny trigger pracy zespołu CS — od tego maila zaczyna się 24-48h licznik aktywacji CSS obiecany userowi na stronie sukcesu (krok A4). Wysyłka to zwykły synchroniczny `Mail::send` bez retry/queue — jeśli ten mail zaginie (spam, błąd SMTP), CS nigdy się nie dowie o zgłoszeniu, a user będzie czekał na aktywację która nigdy nie nadejdzie.

3 CssAgencyWelcomeMail — powitanie agencji + aktywny bonus

Kiedy: `UserRepository::createAccount()` — wysyłany automatycznie przy rejestracji dla `account_type=css_agency`, w tym samym momencie co start trialu.

Do kogo: User (email podany przy rejestracji)

Temat: Twój White Label CSS — co przygotować i co dalej

Rejestracja agencji przyjęta ✓

Nasz zespół skontaktuje się z Tobą, żeby uruchomić **White Label CSS**. Aktywacja przez Google zajmuje zazwyczaj **2-4 tygodnie** od przesłania kompletnych danych.

CO PRZYGOTOWAĆ PRZED KONTAKTEM Z ZESPOŁEM

1. Domena lub subdomena CSS
2. Email domenowy (nie prywatny Gmail)
3. Konto Google (powiązane z emailem domenowym)
4. Logo i nazwa CSS

[[Napisz do nas: css@sembot.com](mailto:css@sembot.com) →]

BONUS: 14 dni Sembota aktywowane

Zanim White Label CSS zostanie aktywowany przez Google (2-4 tygodnie), masz już pełny dostęp do:

- ✓ Pełnej platformy Sembot — kampanie Google, Meta i Microsoft
- ✓ Konfiguracji projektu dla jednego z klientów agencji
- ✓ Feedu produktowego i automatyzacji

[[Zacznij onboarding w Sembocie](#) →]

Pytania? Napisz na css@sembot.com — jesteśmy tu, żeby pomóc.

KONTEKST BIZNESOWY

Trial startuje razem z rejestracją, więc ten mail jest wysyłany od razu — nie czeka na żaden ręczny krok CS. Sekcja bonusu jest deklaratoryjna ("aktywowane"), spójna z tym co user widzi na `/css/agency`.

Tabela plików kluczowych

Plik	Rola
proxy_sembot/internal/handlers/auth.go	ShowCSSPath (/auth/css), ShowSignup/DoSignup (rejestracja + routing po css_type)
proxy_sembot/internal/handlers/css_onboarding.go	ShowShop, SubmitShop, ShowShopDone, ShowAgency
proxy_sembot/internal/handlers/onboarding.go	Start, ShowDomain, SaveDomain, ShowAuditBridge, BridgeStatus, ShowAudit, ResendVerification — wizard Sembota
proxy_sembot/internal/middleware/onboarding_redirect.go	Middleware wypychający nieonboardowanych userów do wizarda
proxy_sembot/internal/middleware/require_project.go	Guard projektu dla /p/{ws}/{proj}/... gdy proj=0
proxy_sembot/internal/views/pages/css_path.templ	Wybór profilu Shop/Agency
proxy_sembot/internal/views/pages/css_onboarding_shop.templ	Formularz sklepu (krok 3 Shop)
proxy_sembot/internal/views/pages/css_onboarding_shop_success.templ	Strona sukcesu Shop
proxy_sembot/internal/views/pages/css_onboarding_agency.templ	Ekran agencji + mailto CTA (Alpine emailClicked)
proxy_sembot/internal/views/pages/onboarding_domain.templ	Wizard krok 1 (Agency) — pole domeny, banner weryfikacji email + resend
proxy_sembot/internal/api/css.go	CSSShopNotify, CSSStartTrial (klient Go → Laravel)
proxy_sembot/internal/api/projects.go	CreateProject, CSSCreateProject (POST /css/shop/project)
sembot-laravel/app/Http/Controllers/Api/Css/CssOnboardingController.php	shopNotify, createShopProject, startTrial
sembot-laravel/app/Http/Controllers/Auth/OAuthController.php	Social login/signup — dziś nie przenosi css_type , patrz "Znane ograniczenia"
sembot-laravel/app/Http/Middleware/HasSubscription.php , Paywall.php	Middleware trialu/paywalla — dziś niepodpięte pod żaden route
sembot-laravel/app/Jobs/Projects/StoreProject.php	Tworzenie projektu — tenant assignment (linia 89, wymaga ≥1 tenanta w landlord DB)
sembot-laravel/app/Notifications/VerifyEmail.php	Generuje link weryfikacyjny {front_url}/auth/verify/{token} — trasa BRAKUJE w proxy_sembot
sembot-laravel/app/Mail/CssShopWelcomeMail.php , CssShopNotifyMail.php , CssAgencyWelcomeMail.php	3 maile transakcyjne — pełna treść w Rozdziale C
locales/pl/css.json , locales/en/css.json	Copy całego flow CSS (UI, nie maile)
proxy_sembot/internal/handlers/signup.go	ShowSignup/DoSignup — czytają middleware.Source(r) / middleware.Locale(r) , przenoszą przez ukryte pole formularza
proxy_sembot/internal/middleware/source.go	WithSource — first-touch capture ?source= w cookie; SanitizeSource
proxy_sembot/internal/middleware/i18n.go	WithLocale — last-touch capture ?lang= w cookie, deterministyczny fallback dla

	niewspieranych kodów
proxy_sembot/internal/middleware/auth.go	ctxSource / ctxLocale , SetSource / Source , SetLocale / Locale — context accessors
proxy_sembot/cmd/server/main.go	middleware.WithSource() / WithLocale() wpięte globalnie w chain
sembot-laravel/app/Listeners/Users/SendRegisterAnalyticsToPostHog.php	Event PostHog user_registered — wzbogacony o language/account_type/source
sembot-laravel/app/Services/PostHog/PostHogService.php	track() — parametr \$userIdentifyOnceData (\$set_once)
sembot-laravel/app/Repositories/UserRepository.php	createAccount() — start trialu + CssAgencyWelcomeMail dla wszystkich kont przy rejestracji, bez wyjątku dla CSS Agency

Znane ograniczenia i otwarte pytania

🚫 Świadomie poza zakresem:

- Formularz zamiast mailto dla zgłoszeń agencji (ryzyko zgubionych leadów, patrz krok B4).
- Interaktywna edukacja platformy przy polu feed URL (kosmetyczny UX).
- Rejestracja przez Google/Facebook/LinkedIn. `/login` ma social buttons, `/signup` nigdy ich nie miał — dotyczy całego produktu, nie tylko ścieżki CSS.
- Ochrona przed duplikatami projektu w `/css/shop`.

Brak egzekwowanego paywalla: w kodzie nie ma dziś nigdzie w produkcie middleware'u faktycznie egzekwującego trial/subskrypcję. User ma pełny dostęp do platformy od razu po rejestracji, niezależnie od statusu trialu. Dotyczy całego produktu, nie tylko CSS.

⚠️ Znane problemy techniczne:

- Trasa `/auth/verify/{token}` (link weryfikacyjny w mailu rejestracyjnym) nie jest zarejestrowana w `proxy_sembot` — dotyczy wszystkich rejestracji przez nowy front. W ścieżce CSS Agency weryfikacja emaila faktycznie blokuje dalszy postęp wizarda (`SaveDomain`), więc tam ten brak ma bezpośredni wpływ.
- Auto-redirect z bridge page (htmx polling) sporadycznie nie triggeruje mimo gotowego `HX-Redirect` po stronie serwera.

Otwarte pytania biznesowe: finalna akceptacja copy emaili (Rozdział C), akceptacja ryzyka mailto jako głównej akcji agencji, aktualność czasów aktywacji CSS (24-48h shop / 2-4 tygodnie agencja), czy maile transakcyjne powinny mieć wariant EN, kiedy `css.sembot.com` uruchomi kolejne wersje językowe UE poza dzisiejszymi PL/EN (wzorec URL już gotowy, patrz "Analityka i atrybucja").